



BigQuery Connector — Credential Provisioning Guide

Version 0.1.0, 2026-07-29

Table of Contents

Overview.....	1
What kind of account to create.....	1
Permissions to grant	1
Generating the credential.....	2
BigQuery API enablement	2
Mapping values into Crogl.....	3
Verifying the setup.....	3

Overview

The Crogl BigQuery connector reads and queries data in a Google Cloud project's BigQuery datasets. There are two ways it can authenticate, and the customer's environment determines which one applies:

- **Application Default Credentials (ADC)**—if the machine or service running Crogl is itself a Google Cloud workload (a Compute Engine VM, a GKE pod, a Cloud Run service, etc.) with a service account already attached to it, no key file is needed at all. Leave the `service_account_json` field empty.
- **A service account JSON key**—if Crogl runs anywhere else (on-prem, another cloud, a customer's own server), a customer GCP administrator must create a service account, grant it the roles below, generate a JSON key, and hand that key to Crogl.

These steps are performed by whoever administers the customer's **Google Cloud project** (IAM & Admin access), not a Crogl role.

What kind of account to create

Create a dedicated **GCP service account** (not a human user's own credentials):

1. In the Google Cloud Console, go to **IAM & Admin > Service Accounts**.
2. Click **Create service account**.
3. Enter a **service account name** (e.g. `crogl-bigquery-reader`)—the console auto-generates an editable **service account ID** from this name. Note: the ID cannot be changed after the account is created, so review it before continuing.
4. Optionally add a description, then click **Create and continue** to proceed to role assignment (or **Done** to skip and grant roles afterward from the Service Accounts list).

Permissions to grant

IMPORTANT

`roles/bigquery.user` alone is **not** sufficient for this connector, even though its name suggests general BigQuery access. Per Google's own IAM role reference, `roles/bigquery.user` grants `bigquery.jobs.create` and the ability to create new datasets (becoming their owner)—but it does **not** include `bigquery.tables.getData`, so it cannot read data in datasets/tables it doesn't already own. A service account with only this role will authenticate successfully and then fail to read any existing customer data.

Grant **both** of the following predefined roles:

Role	What it grants (per Google's IAM reference)
<code>roles/bigquery.dataViewer</code> ("BigQuery Data Viewer")	Read access to table/view data and metadata (<code>bigquery.tables.getData</code> , <code>bigquery.datasets.get</code>). Grant this on the specific dataset(s) the connector needs, or at the project level for broader access.
<code>roles/bigquery.jobUser</code> ("BigQuery Job User")	Permission to run query jobs in the project (<code>bigquery.jobs.create</code>). Grant this at the project level — without it, the service account can see data but cannot execute a query against it.

Grant these either during the **Create and continue** role-assignment step above, or afterward via **IAM & Admin > IAM > Grant Access**, entering the service account's email as the principal.

Generating the credential

NOTE

Skip this entire section if the customer's Crogl deployment runs as a workload on Google Cloud (GCE, GKE, Cloud Run, etc.) with this service account attached directly to that resource — Application Default Credentials will pick it up automatically with no key file. Only generate a JSON key if Crogl runs outside Google Cloud.

1. In **IAM & Admin > Service Accounts**, select the service account created above.
2. Open the **Keys** tab.
3. Click **Add key > Create new key**.
4. Select **JSON** as the key type and click **Create**.
5. The key file downloads automatically — **this is the only time it can be downloaded**; if it's lost, delete the key and create a new one rather than searching for a way to retrieve it.

IMPORTANT

Google enforces the organization policy constraint `iam.disableServiceAccountKeyCreation` by default on any GCP organization created on or after May 3, 2024. If this policy is in effect, the **Create new key** step above will fail — the console shows "Service account key creation is disabled," and the API/CLI return `FAILED_PRECONDITION`. If the customer hits this, their GCP organization admin must grant a policy exception for this project/service account before a JSON key can be created at all. This is increasingly likely to affect newer customer GCP organizations, so check for it early rather than assuming key creation will simply work. Google's own guidance is to prefer Workload Identity Federation over long-lived JSON keys where possible — worth raising with the customer's GCP admin as an alternative if they're security-conscious about static keys.

BigQuery API enablement

The BigQuery API is automatically enabled on new GCP projects. On an older or previously-restricted project, confirm it's enabled: **APIs & Services > Library**, search for "BigQuery API", and click **Enable** if it isn't already. If queries fail with an API-not-enabled error, this is the first thing to

check.

Mapping values into Crogl

Crogl field	Where it came from	Notes
<code>project_id</code>	The GCP project ID shown in the Cloud Console (top project selector, or IAM & Admin > Settings)	Required. This is the project containing the datasets to query — not necessarily the project the service account itself lives in, if cross-project access was granted.
<code>default_dataset</code>	Customer's choice — the dataset name most commonly queried, if they want to allow unqualified table references	Optional. Leave empty if every query will fully qualify <code>dataset.table</code> .
<code>dashboard_tickets_table</code>	Customer's choice — the fully-qualified BigQuery table (dataset.table) that backs the ticket-dashboard cards	Optional. Leave empty if ticket-dashboard cards are not needed. The table must have columns: <code>ticket_id</code> , <code>created_at</code> , <code>resolved_at</code> , <code>assigned_to</code> , <code>title</code> , <code>severity</code> .
<code>service_account_json</code>	Keys tab > the downloaded JSON key file's contents	Optional — leave empty entirely if Application Default Credentials apply (see Overview). Treat as a password: never paste it into chat, a ticket, or this document.

Verifying the setup

- **In Crogl:** run a minimal query against the connector (e.g. `bigquery.py query --sql 'SELECT 1' --limit 1`) and confirm it returns a result rather than an error.
- **If auth fails outright** (credential rejected before reaching BigQuery): the JSON key is malformed, truncated, or was regenerated/deleted after being copied — re-download a fresh key.
- **If you get a permission-denied error mentioning `bigquery.jobs.create:roles/bigquery.jobUser`** is missing at the project level.
- **If you get a permission-denied error mentioning `bigquery.tables.getData`** (or the query runs but returns no columns/rows it should have access to): `roles/bigquery.dataViewer` is missing on the relevant dataset(s) — this is the specific failure mode of granting only `roles/bigquery.user`, see the warning above.
- **If the API itself appears unreachable:** confirm the BigQuery API is enabled on the project (see above).
- **Zero rows back is expected** on a query whose **WHERE** clause matches nothing — that's a successful, empty result, not an error.